DesertRunner: An Endless 3D Desert Runner Game Developed in Unity and C#

Project Report – Game Mechanics and Game Design & Development

Submitted by: [Student Name], [Roll Number] Course: [Course Name] Under the guidance of: [Professor Name] Department of Computer Science and Engineering, [College Name] Academic Year: [Year]

Chapter 1: Introduction to Game Design and Development

1.1 Overview of Game Design

Digital game design is the discipline of defining the rules, goals, challenges and feedback loops that make an interactive experience engaging. A game is a system in which the player makes decisions, the system responds, and the cycle repeats. Game mechanics are the individual rules and actions inside that system, such as moving, jumping, collecting items or avoiding enemies. Game dynamics are the behaviours that appear when those mechanics interact over time, and the aesthetics are the emotions the player feels as a result.

1.2 The Game Development Lifecycle

Game development follows a lifecycle similar to general software engineering but with added creative stages: concept and pre-production, design documentation, prototyping, production (assets and programming), integration, testing and debugging, optimisation, and release. DesertRunner followed this lifecycle in a compact form. The concept was chosen first, the core mechanics were prototyped, the desert world and assets were added, and the finished game was tested and tuned.

1.3 Purpose of the Project

DesertRunner (shown in-game as DESERT DASH) is a three-lane endless runner set in a sun-baked desert canyon. The player rides forward automatically on a sandboard while the speed slowly increases, and must switch lanes, jump over dry logs, slide under stone arches and avoid sandstone boulders and charging coyotes. Coins increase the score, and two power-ups, a Magnet and a Shield, give temporary advantages. The purpose of the project is to apply game mechanics, level design, scripting and interface design in a complete, playable Unity game.

1.4 Objectives

- To design and implement a complete endless-runner game in Unity using C#.
- To implement core mechanics: lane switching, jumping, sliding, collision, collecting and scoring.
- To create a recycled 3D desert environment that never ends without increasing memory use.
- To apply progressive difficulty so the game gets harder the longer the player survives.
- To build a usable interface with a main menu, pause screen, game-over screen and live score display.
- To integrate audio and particle effects that respond to gameplay events.
- To support both keyboard and touch controls.

1.5 Significance and Scope

Endless runners are among the most popular casual and mobile game genres because the rules are simple but replay value is high. Building one covers many core topics at a manageable size: input, state management, object pooling, collision detection, procedural content, audio and user interface. The scope is a single continuous level for one player on PC, with a design that also supports touch devices. Multiplayer, online leaderboards and a story mode are outside the current scope and are listed as future work.

1.6 Expected Outcomes and Practical Applications

The expected outcome is a stable, playable game with clear controls, fair difficulty and a best score saved between sessions. The techniques used, such as object pooling, state machines and procedural generation, apply to mobile games, simulations, training applications and interactive visualisation.

Chapter 2: Literature Review and Existing Game Systems

2.1 Existing Endless Runner Games

The endless runner genre was popularised by Temple Run, Subway Surfers and Jetpack Joyride. These games share an automatic forward movement, a few player actions, obstacles to avoid, collectible currency, power-ups and a score that encourages replay. Subway Surfers uses three lanes, swipe controls, jumping and rolling, which is the structure DesertRunner follows.

2.2 Game Engines

Game engines provide rendering, physics, audio, input and tools so that developers can concentrate on gameplay. Unity, Unreal Engine and Godot are the most widely used. Unity was chosen because it uses C#, has a large learning community, supports 2D and 3D, offers the Universal Render Pipeline for efficient rendering and can build for desktop, mobile and web from one project.

2.3 Relevant Techniques

Technique	Use in endless runners	Use in DesertRunner
Object pooling	Reuse obstacles and scenery instead of creating and destroying them	Cacti, rocks, logs, arches, coyotes and coins are pooled and reused
Track recycling	Move passed track sections to the front	Six segments of 40 units are recycled continuously
State machines	Control menu, play, pause and game-over behaviour	Four states: Menu, Play, Pause and Dead
Procedural patterns	Generate obstacle layouts at run time	Obstacle and coin patterns are chosen randomly with difficulty rules
Procedural assets	Create content from code instead of imported files	Models, sounds and music are generated by script

2.4 Limitations in Existing Implementations

Many beginner endless-runner projects share the same weaknesses. They create and destroy objects constantly, which causes memory spikes and frame drops. They depend on large asset packs, which makes projects heavy and hard to reproduce. Random obstacle layouts can produce impossible situations. They often support only one input method.

2.5 How This Project Addresses Them

DesertRunner reuses all gameplay objects through pools, so objects are not created or destroyed during normal play. All models, sounds and music are generated from code, so no external assets are required, and the generated objects can be saved as real prefabs

and materials, or replaced with custom models. Obstacle patterns always leave at least one free lane. Keyboard, touch swipes and an on-screen pause button are all supported.

Chapter 3: Game Concept and Project Planning

3.1 Game Concept

DesertRunner places the player in a hot desert canyon with sand dunes, mesas and saguaro cacti. The character rides a sandboard forward automatically. The goal is to survive as long as possible and reach the highest score. There is no fixed ending, which gives the game its replay value.

3.2 Genre, Storyline and Objectives

- Genre: 3D endless runner, casual arcade.
- Setting: a desert trail with warm light, drifting sand and a hazy sky.
- Storyline: a rider races across the desert, escaping coyotes and dodging the hazards of the trail. The story is deliberately light because the genre depends on gameplay.
- Gameplay objective: travel as far as possible, collect coins, use power-ups wisely and beat the saved best score.

3.3 Target Audience

The game targets casual players of all ages who enjoy short, repeatable sessions. Controls are simple enough to learn in seconds, while rising speed gives challenge to experienced players.

3.4 Technical Requirements

Item	Requirement
Game engine	Unity (Universal Render Pipeline, 3D template)
Programming language	C#
Input	Keyboard, or touch swipes on mobile
Platform	Windows PC (also buildable for Android and WebGL)

Item	Requirement
Hardware (minimum)	Dual-core CPU, 4 GB RAM, integrated graphics
Tools	Visual Studio or VS Code, Git for version control

3.5 Game Design Document Summary

Element	Description
Title	DesertRunner (shown in-game as DESERT DASH)
Camera	Third person, behind and above the player, looking forward
Lanes	Three lanes, each 2.2 units wide
Starting speed	12 units per second, increasing to a maximum of 28
Player actions	Move left, move right, jump, slide
Obstacles	Sandstone boulders, dry logs, stone arches, coyotes
Collectibles	Coins, Magnet power-up (8 seconds), Shield power-up (12 seconds)
Score	Distance travelled plus 10 points per coin
Lose condition	Colliding with an obstacle while not shielded

3.6 Development Schedule and Milestones

Week	Milestone
1	Concept, genre study and game design document
2	Project setup, track generation and player movement
3	Obstacles, collision handling, scoring and game states

Week	Milestone
4	Coins, power-ups, coyotes and difficulty progression
5	Desert environment, drifting sand, audio and effects
6	User interface, prefab export and scene-object support
7	Testing, balancing and optimisation
8	Documentation, demonstration and final submission

Adjust the weeks above to match your actual project timeline.

Chapter 4: Game Mechanics Design

4.1 Player Movement and Lanes

The player always moves forward, and the game controls the speed. The player only chooses which of three lanes to occupy. Pressing A or Left moves one lane left, and D or Right moves one lane right. The horizontal position moves smoothly toward the target lane instead of snapping, which feels natural. On touch devices a horizontal swipe does the same.

4.2 Jumping

Pressing W, Up or Space (or swiping up) makes the player jump if they are on the ground. The jump gives an upward velocity of 10 units per second and gravity pulls the player back at 32 units per second squared, so the jump is quick and responsive. Jumping is used to clear low obstacles such as dry logs.

4.3 Sliding

Pressing S, Down or swiping down starts a slide of 0.7 seconds, during which the player model is lowered so the player can pass under stone arches. If the player slides in the air, they are pulled quickly to the ground first, which allows a fast reaction to an incoming arch.

4.4 Collision Responses

Collision is evaluated against each active obstacle near the player. The rules for each type are:

- Sandstone boulder: always blocks the lane. The player must change lanes.
- Dry log: blocks the player only if they are near the ground. Jumping avoids it.
- Stone arch: blocks the player unless they are sliding. Sliding avoids it.
- Coyote: runs toward the player. The player must change lanes.

On a hit, the screen shakes, a crash sound plays and the game moves to the Dead state, unless the Shield absorbs the hit.

4.5 Object Collection and Scoring

Coins are collected by touching them, which adds to the coin count, plays a sound and spawns a small particle burst. The score is the distance travelled plus ten points for each coin. The best score is saved with PlayerPrefs so it persists between sessions.

4.6 Power-ups

Power-up	Duration	Effect
Magnet	8 seconds	Coins within range are pulled toward the player automatically
Shield	12 seconds	Absorbs one collision, then disappears; an aura shows under the player

4.7 Health Management

DesertRunner uses a one-hit health model, which is standard for endless runners. The Shield acts as a temporary extra life: it absorbs one collision and removes the obstacle that was hit so the player can continue.

4.8 Enemy Interaction

Coyotes are the only moving enemy. They appear only once the speed is above 15 and then spawn with a 30 percent chance. They run toward the player faster than the scrolling track, so their effective approach speed is higher than that of static obstacles. This forces the player to react early and change lanes.

4.9 Difficulty Progression

Speed starts at 12 units per second and increases by 0.25 per second up to a maximum of 28. Above a speed of 15, harder patterns appear that combine a coyote with a boulder, and coin rows become shorter. The result is a continuous increase in challenge without separate levels.

4.10 Winning and Losing Conditions

There is no winning state because the game is endless. The aim is a higher score than before. The game is lost when the player collides with an obstacle without a Shield. The game-over screen then shows the score and best score, and the player can restart immediately.

Chapter 5: Game Environment and Level Design

5.1 Track Construction

The track is made of six identical segments, each 40 units long. A segment contains a sandy road surface, lane markings in the form of wheel ruts, raised dune banks on both sides, distant dunes and mesas, and a base layer under the scene. When the player passes a segment, it is moved to the front of the line and refilled with new content. This produces an infinite track from six segments.

5.2 Environment Elements

- Terrain: a sandy road with raised dune banks on either side.
- Scenery: saguaro cacti on both banks, with more cacti close to the track and fewer further away.
- Background: sand dunes and flat-topped mesas, with a warm hazy sky.
- Weather: drifting sand particles that blow around the player.
- Lighting: a warm directional sun with soft shadows and a warm ambient colour.

5.3 Obstacles and Interactive Objects

Obstacles are placed in groups called patterns. A pattern can be a single blocker, a pair of blockers with one open lane, a full row of logs that must be jumped, or a full row of arches that must be slid under. Coins are placed in lines, often leading toward the safe lane, so collecting them also guides the player.

5.4 Fairness of Level Design

Every obstacle pattern guarantees at least one possible route. When two lanes are blocked, the third is open, and full rows use only obstacles that can be jumped or slid under. This prevents unfair deaths and keeps the game enjoyable at high speed.

5.5 Influence of Level Design on Difficulty and Engagement

Difficulty depends on speed and pattern complexity together. At low speed the player has time for several lane changes between patterns, while at high speed the same layout demands fast reactions. Coin lines reward risk because the best coins are often next to obstacles. The regular rhythm of cacti along the track also gives the player a strong sense of speed.

5.6 Checkpoints

As an endless game, DesertRunner has no checkpoints. The distance counter shows continuous progress, and the saved best score is a persistent target.

Chapter 6: Game Assets and Character Design

6.1 Asset Approach

DesertRunner uses a code-generated asset approach. Instead of importing sprites or models, every 3D object is assembled at run time from Unity primitives such as cubes, spheres, cylinders and capsules, each with a simple coloured material. This keeps the project small, removes licensing issues and keeps the visual style consistent.

6.2 Character Design

The player is built from primitives: a sandboard, legs, a red shirt, arms, a head, a wide-brimmed hat and a teal scarf. A glowing disc appears under the character while the Shield is active. The body is placed in a Model object so that it can be lowered during a slide without affecting the collision position.

6.3 Obstacle and Enemy Models

Object	Construction
Sandstone boulder	Large rock-coloured sphere with a darker cap and smaller rocks beside it
Dry log	Horizontal wood-coloured cylinder with broken branch stubs
Stone arch	Two stone pillars and a flat beam across the top
Coyote	Body, head, snout, ears, tail and four legs from cubes, facing the player
Saguaro cactus	Tall green trunk with curved arms built from blocks
Coin	Thin gold cylinder that rotates continuously
Magnet	Red horseshoe made from three blocks with white tips
Shield	Cyan sphere with a white ring

6.4 Materials, Lighting and Rendering

Materials use the Universal Render Pipeline Lit shader with low smoothness for a matte look, and are shared between objects of the same colour so rendering stays efficient. A warm directional light creates soft shadows and a warm ambient light fills dark areas. Fog uses the same colour as the sky so that distant dunes fade into the haze. Visual consistency comes from a limited palette of sand, orange and cactus green, with gold and red reserved for collectibles and danger.

6.5 Importing, Scaling and Saving Assets

In the editor the game can save every generated object as a real prefab and material file in the Assets folder. These prefabs can then be edited, or the developer can paste their own models into the scene and the game will use them in place of the generated ones. All models are authored at the same scale, where one lane is 2.2 units wide, so replacement models only need to respect that size.

6.6 Animation of Assets

Simple animation is done in code. Coins spin, the player model lowers during a slide, the camera shakes on impact, and particles drift, burst and fade. This gives lively feedback without animation clips.

Chapter 7: Game Architecture and System Design

7.1 Overall Architecture

DesertRunner uses a single-controller architecture. One C# script, DesertRunner, is attached to an empty GameObject. When the game starts, the script builds the entire scene under a root object called World, so the scene itself stays almost empty and the whole game can be reproduced from one component. This keeps the project easy to set up and easy to explain.

Layer	Responsibility	Main elements
Input layer	Reads keyboard and touch input and converts it to actions	ReadInput, Swipe, pause button hit test
Game logic layer	Updates state, speed, player physics, collisions and score	Update loop, State machine, Score, Die, NewRun
World layer	Builds and recycles track, scenery, obstacles and pickups	Track segments, pools, Fill, Add
Presentation layer	Draws models, particles, camera motion and the interface	Generated models, drifting sand, popup effects, OnGUI
Audio layer	Generates and plays sound effects and music	Clip generator, Sfx, music source
Persistence layer	Saves the best score and exports prefabs in the editor	PlayerPrefs, ExportAll

7.2 Main Components and Data Structures

- Seg: a track segment with a root transform, a container for dynamic objects, and a list of cacti.

- Item record: stores an obstacle or pickup, its type, lane, distance, and whether it has been collected or hit.
- Pools: a dictionary of stacks, one per object type, from which objects are taken and to which they are returned.
- Player: a transform with a Model child and an Aura child for the shield.
- Particle systems: drifting sand, coin pop, shield burst, dust puff and board spray.

7.3 Game State Diagram (described)

The game is controlled by four states.

From	Event	To
Menu	Tap, Space or Jump pressed	Play
Play	Pause key (P or Esc) or on-screen button	Pause
Pause	Pause key, jump or tap	Play
Play	Collision without Shield	Dead
Dead	Tap or Space after half a second	Play (new run)

7.4 Gameplay Flowchart (described)

1. Start the game and show the menu.
2. When the player starts, reset speed, score, coins, power-ups and world.
3. Each frame: read input, increase speed, update lane position, jump and slide.
4. Scroll the world: recycle any segment the player has passed and refill it.
5. Check each nearby obstacle and pickup for collisions.
6. If a coin or power-up is touched, apply its effect and play feedback.
7. If an obstacle is hit, check for a Shield; if none, go to the Dead state.
8. Update the score and the interface and return to step 3.

7.5 Class and Component Relationships

Because the game is built from one controller class, the relationships are between its internal parts rather than between many classes. The controller owns the pools, segments, player and effects. Segments own the objects spawned into them. The pool

owns objects that are not in use. Particle systems are children of World and are repositioned each frame around the player. This one-way ownership prevents circular dependencies and makes cleanup simple.

7.6 Communication Between GameObjects and Scripts

In this design, individual GameObjects do not have their own scripts. The controller holds direct references to the transforms it needs and changes their position, rotation, scale and active state each frame. This avoids the overhead of message passing and of hundreds of separate Update calls, since one Update call drives everything, and it makes the execution order completely predictable.

Chapter 8: Game Development Using Unity and C#

8.1 Unity Installation and Project Setup

1. Install Unity Hub and a recent Unity editor version with the 3D (URP) template.
2. Create a new project named DesertRunner.
3. Open the main scene. The default Main Camera and Directional Light can stay, since the script uses and adjusts them.
4. Create a new C# script named DesertRunner and paste the code into it.
5. Create an empty GameObject, attach the script, and press Play.

8.2 Scene Development and GameObject Creation

On Start the script creates a World root, a Pool root, six track segments, the player, the particle systems, the audio sources and the camera settings. All objects are created through code with helper functions that build Unity primitives, remove their default colliders, assign a shared material and parent them correctly. Removing colliders is deliberate: the game does its own collision test, so colliders would only waste processing.

8.3 Components Used

Component	Purpose in DesertRunner
Transform	Position, rotation and scale of every object
MeshFilter and MeshRenderer	Display the primitive shapes with materials

Component	Purpose in DesertRunner
Camera	Third-person view with shake and smooth following
Light	Warm directional sun with soft shadows
AudioSource	Music and sound-effect playback
ParticleSystem and ParticleSystemRenderer	Drifting sand, bursts and spray

8.4 Prefabs

Prefabs let the game replace generated models with custom ones. Each model type has an optional slot on the script. If a slot is empty, the game checks for a matching object in the scene, then for a saved prefab in the Assets folder, and finally generates the model in code. In the editor, a built-in export step saves every generated object as a prefab with its materials, so the whole game can be turned into editable assets with one Play press.

8.5 Scripting and Input Handling

The game supports Unity's new Input System and the older Input Manager through conditional compilation. Keyboard keys are read once per press for lane changes, jump, slide, pause and mute. For touch, a swipe is detected from the start and end of a finger drag, with a minimum distance of 60 pixels, and is converted to left, right, up or down. A small pause button in the corner can be tapped on any device.

8.6 Gameplay Logic

The update loop applies a fixed set of steps: input, speed increase, lane smoothing, jump physics, slide timer, power-up timers, world scrolling, collision checks, particle movement, camera update and score calculation. The speed rule is simple: speed = min(28, speed + 0.25 x deltaTime). Because all motion is multiplied by deltaTime, the game behaves the same at any frame rate.

8.7 Object Pooling and Track Recycling

When the player passes a segment, the script returns every object in it to the pool, moves the segment to the far end of the track and fills it with a new pattern. When a new object is needed, it is taken from the pool, and built only if the pool is empty. After a short warm-

up, the game creates no new objects, so memory use stays flat and the garbage collector does not cause stutters.

Chapter 9: Physics, Animation, and Collision Detection

9.1 Design Choice: Custom Kinematic Physics

Unity provides a full physics engine with Rigidbody and Collider components, but an endless runner does not need realistic physics. Movement must be exact, predictable and cheap. DesertRunner therefore uses custom kinematic physics. The player is moved by script, and only a few quantities are simulated: vertical velocity and gravity for jumping, and smooth interpolation for lane changes. No Rigidbody is used.

9.2 Gravity and Jumping

On takeoff the vertical velocity is set to 10 units per second. Each frame, gravity of 32 units per second squared is subtracted from the velocity and the velocity is added to the height. When the height returns to zero, the player lands and the velocity is reset. This produces a jump of about 1.5 units in height and about 0.6 seconds in duration, which is long enough to clear a log and short enough to land before the next obstacle.

9.3 Collision Detection

Each obstacle stores its lane and its distance along the track. Every frame, only obstacles near the player are tested. The test compares the distance difference along the track with a hit depth that depends on the obstacle type, and compares the horizontal difference with a lane-width tolerance. The vertical rules are then applied: logs are cleared by height, arches are cleared by sliding, and boulders and coyotes cannot be cleared. This is an axis-aligned distance test, which is much cheaper than mesh collision and exact enough for lane-based gameplay.

9.4 Moving Enemies

Coyotes carry their own distance value, which moves toward the player at the track speed plus 16 units per second. The same collision test is then used, so no special physics is needed for moving enemies.

9.5 Comparison With Unity Physics

Aspect	Unity Rigidbody and Collider	DesertRunner approach
Cost per object	Higher; each collider is simulated	Very low; one comparison per nearby object
Predictability	Depends on the physics step and contacts	Fully deterministic
Tuning jump feel	Needs mass, drag and gravity settings	Direct control of velocity and gravity
Best used for	Realistic bouncing, stacking, vehicles	Fast lane-based runners

If the game were extended, for example with ramps or physical props, Rigidbody and trigger colliders could be added to those objects, and trigger events such as OnTriggerEnter could replace the manual tests for them.

9.6 Trigger-Based Events and Animation

Although Unity triggers are not used, the game uses the same event idea. Touching a coin triggers a sound, a particle burst, an increase of the coin count and removal of the coin. Touching a power-up starts its timer. Hitting an obstacle triggers a crash sound, a screen shake and the Dead state. Animation is procedural rather than clip-based: coins rotate, the player model scales down while sliding, the camera shakes after a crash, and particles drift and fade. An Animator Controller could be added in future for a rigged character.

Chapter 10: User Interface and User Experience Design

10.1 Interface Overview

The interface is drawn with Unity's immediate mode GUI. All sizes are multiplied by a scale factor based on screen height, so the text and buttons look the same on a phone and on a large monitor.

10.2 Screens

Screen	Contents
Main menu	Game title DESERT DASH, the prompt Tap or Space to start, the saved best score and a line listing the controls
Gameplay HUD	Live score, coin count, active power-up timers for Magnet and Shield, and a pause button
Pause screen	PAUSED title and a prompt to resume with P, Esc or a tap
Game-over screen	WIPEOUT text, final score, best score and a prompt to restart

10.3 Score and Power-up Indicators

The score and coin count are always visible at the top of the screen. When a power-up is active, a coloured label with a countdown in seconds appears below them: red for Magnet and blue for Shield. A glowing aura under the player gives a second visual cue for the Shield.

10.4 Pause, Restart and Mute

The game can be paused with P, Esc or the on-screen button. While paused, music is paused and gameplay is frozen. After a crash, a half-second delay prevents an accidental instant restart, and then a tap or Space starts a new run immediately. The M key mutes or unmutes all audio.

10.5 Usability Considerations

- Controls are explained on the menu screen and need no tutorial.
- Restart is immediate, which suits short, repeated sessions.
- Text is large and high-contrast against the background.
- Touch controls use swipes, which are familiar from other mobile runners.
- Feedback is multisensory: sound, particles and screen shake all signal important events.

Chapter 11: Audio and Visual Effects Integration

11.1 Procedural Audio

All audio is generated in code by filling an array of samples with mathematical waveforms and wrapping it in an AudioClip. This means the project contains no audio files. Each effect uses a different design: jump is a rising sine sweep, coins are short bright tones, power-ups are a rising chime, the slide is a short noise swish and the crash is a low noise burst with a fast decay.

11.2 Background Music

The music is a short looping melody and bass line with a desert character, built on a phrygian-dominant scale in A, which gives the typical Middle-Eastern desert sound. It starts when the first run begins, pauses with the game and can be muted at any time.

11.3 Sound Effect Table

Event	Sound
Jump	Short rising sweep
Slide	Soft noise swish
Coin collected	Bright high tone
Power-up collected	Rising chime
Collision	Low crash
Game start	Power-up chime

11.4 Particle Effects

- Drifting sand: a box-shaped emitter that follows the player and blows fine sand particles through the air.
- Coin pop: a small burst when a coin is collected.
- Shield burst: a larger burst when the Shield is picked up.
- Dust puff and board spray: effects that appear on landing and while riding, giving a sense of speed and contact with the sand.

11.5 Lighting and Atmosphere

A warm directional light with soft shadows sets the mood, and a matching warm ambient colour and fog colour make the distant dunes fade into a hazy sky. The camera shakes briefly on impact, which adds weight to a crash without needing any animation clip.

11.6 Synchronisation of Sound, Effects and Gameplay

Sound and particles are triggered at the exact point in the update loop where the gameplay event is detected, so there is no delay between what the player sees and what they hear. For example, when a coin is collected, the coin counter, the sound and the burst all happen in the same frame.

Chapter 12: Gameplay Implementation and Integration

12.1 Integration of Subsystems

The individual systems described in earlier chapters were developed and tested separately and then integrated into one update loop. The track generator supplies the obstacles and coins, the player controller moves through them, the collision system reports events, and the score, audio, particle and interface systems respond to those events. Because all subsystems are driven from one controller, integration problems such as event ordering were easy to detect and fix.

12.2 A Typical Gameplay Sequence

1. The menu appears with the saved best score.
2. The player taps or presses Space and the music starts.
3. The character rides at 12 units per second and the first patterns appear.
4. The player changes lanes, jumps logs and slides under stone arches while collecting coins.
5. Speed rises, coyotes appear and patterns become denser.
6. The player collects a Shield, which later absorbs a collision.
7. A second collision ends the run, the screen shakes and the game-over screen appears.
8. If the score beats the best score, the new best is saved and shown.
9. The player restarts and the cycle repeats.

12.3 Scene Transitions

DesertRunner uses a single scene. Transitions between menu, play, pause and game-over are state changes, not scene loads, so they are instant and need no loading time. Restarting resets the speed, score, coins, power-ups and the world, and returns every object to the pool.

12.4 Level Completion and Player Progression

Because the game is endless, progression is measured by distance and score rather than level completion. The player improves through repeated runs, learning the obstacle patterns and the timing of jumps and slides, and is motivated by the saved best score.

12.5 Support for Custom Models

The integrated game can use models supplied by the developer. If objects with the expected names, such as Cactus, Rock, Log, StoneArch, Coyote, Coin, Player or TrackSegment, are placed in the scene, the game hides them and uses them as templates. If several objects share a name, one is picked at random for each spawn, which adds visual variety. Models that are not supplied are generated by the game as before, so replacement can be done gradually.

12.6 Overall Game Execution

The execution flow is: Start builds the world and effects, then the menu waits for input. Update runs every frame during play, LateUpdate positions the camera and the particle emitters, and OnGUI draws the interface. The run ends in the Dead state and can be restarted without reloading anything.

Chapter 13: Game Testing, Debugging, and Optimization

13.1 Testing Approach

Testing combined manual play testing with targeted functional tests. Each mechanic was tested alone first, then in combination with others, and finally in long play sessions to find rare problems. The tables below describe the test cases. The Result column should be filled in during testing.

13.2 Functional and Input Test Cases

No.	Test	Expected result	Result
1	Press A or Left	Player moves one lane left, stops at the edge	[Pass/Fail]
2	Press D or Right	Player moves one lane right, stops at the edge	[Pass/Fail]
3	Press Space, W or Up on the ground	Player jumps and lands	[Pass/Fail]
4	Press Space in mid-air	No double jump occurs	[Pass/Fail]
5	Press S or Down on the ground	Player slides for about 0.7 seconds	[Pass/Fail]
6	Press S in mid-air	Player drops quickly to the ground and slides	[Pass/Fail]
7	Swipe left, right, up, down on touch	Same action as the matching key	[Pass/Fail]
8	Press P or Esc during play	Game and music pause	[Pass/Fail]
9	Press M	All audio is muted and unmuted	[Pass/Fail]

13.3 Collision and Gameplay Test Cases

No.	Test	Expected result	Result
10	Run into a sandstone boulder	Game over	[Pass/Fail]
11	Jump over a dry log	No collision	[Pass/Fail]
12	Run into a log without jumping	Game over	[Pass/Fail]
13	Slide under a stone arch	No collision	[Pass/Fail]
14	Run into an arch without sliding	Game over	[Pass/Fail]
15	Run into a coyote	Game over	[Pass/Fail]
16	Collect a coin	Coin count and score increase, sound plays	[Pass/Fail]

No.	Test	Expected result	Result
17	Collect a Magnet	Nearby coins are pulled in for 8 seconds	[Pass/Fail]
18	Collect a Shield, then hit an obstacle	Hit is absorbed once, Shield ends	[Pass/Fail]
19	Beat the best score and restart	New best is saved and shown	[Pass/Fail]

13.4 UI Test Cases

No.	Test	Expected result	Result
20	Start the game	Menu shows title, best score and controls	[Pass/Fail]
21	Game over	Score and best score are displayed	[Pass/Fail]
22	Resize the window or use another resolution	Text stays readable and correctly placed	[Pass/Fail]
23	Tap the pause button	Game pauses and resumes correctly	[Pass/Fail]

13.5 Gameplay Balancing

Balancing focused on three values: the speed growth rate, the jump and gravity values, and the density of obstacle patterns. If speed grew too fast, new players could not survive the first minute, and if it grew too slowly, experienced players got bored. The values of 12 starting speed, 0.25 increase per second and 28 maximum were chosen after play testing to give a gentle start and a challenging late game. Coyotes only appear above a speed of 15 so that new players meet them only after learning the basic controls.

13.6 Bugs Identified and Fixed

Replace the examples below with the real bugs you found during development.

Bug	Cause	Fix
Objects reappeared at the origin after being hidden	Pooled objects were not reset when reused	Reset position, rotation and scale whenever an object is taken from the pool
Impossible obstacle layouts at high speed	Random patterns could block all three lanes	Patterns now always leave at least one free lane
Pause button also triggered a lane change	Touch input was read twice	Button taps are checked before swipes
Frame drops during long runs	Objects were created and destroyed constantly	Object pooling and track recycling

13.7 Performance Optimisation

- Object pooling and track recycling: no object creation after warm-up.
- No colliders on generated objects: collision is done by a cheap distance test.
- Shared materials: objects with the same colour share one material.
- Single update loop: one script drives all objects instead of hundreds of Update calls.
- Particle limits: each particle system has a maximum particle count.
- Fog and a limited draw distance: hides the end of the world and avoids rendering far objects.

13.8 Frame Rate and Memory Monitoring

Frame rate and memory were observed with the Unity Profiler and the Stats window. The measured values should be recorded in Chapter 14 together with the test machine specification.

Chapter 14: Results and Performance Evaluation

14.1 Final Implementation

The completed DesertRunner is a fully playable endless runner. It includes lane switching, jumping, sliding, five obstacle and pickup types, two power-ups, progressive difficulty, a saved best score, a complete interface, procedural audio, drifting sand and effects, keyboard and touch input, and editor tools for exporting and replacing models.

14.2 Screenshots

Insert the following screenshots here:

- Figure 14.1: Main menu with the DESERT DASH title and best score.
- Figure 14.2: Gameplay on the desert trail with cacti and dunes.
- Figure 14.3: Player jumping over a log.
- Figure 14.4: Player sliding under a stone arch.
- Figure 14.5: Shield and Magnet active, with timers visible.
- Figure 14.6: Game-over screen.
- Figure 14.7: The Hierarchy window showing the generated World object.

14.3 Performance Results

Record your own measurements in the table below.

Metric	Measured value
Test machine (CPU, GPU, RAM)	[fill in]
Average frame rate at start	[fill in] FPS
Average frame rate at maximum speed	[fill in] FPS
Memory usage after 5 minutes	[fill in] MB
Objects created after warm-up	0 (all objects are reused)
Build size	[fill in] MB

A line graph of frame rate over time from the Unity Profiler can be added here to show that performance stays stable as speed increases.

14.4 Player Feedback

Feedback should be collected from a small group of players. A suggested format is a short form that asks players to rate controls, difficulty, visuals and enjoyment from 1 to 5 and to give comments. Summarise the averages and the main comments here.

Aspect	Average rating (1 to 5)	Main comment
Controls	[fill in]	[fill in]

Aspect	Average rating (1 to 5)	Main comment
Difficulty	[fill in]	[fill in]
Visuals	[fill in]	[fill in]
Enjoyment	[fill in]	[fill in]

14.5 Comparison With the Original Objectives

Objective	Status
Complete endless runner in Unity and C#	Achieved
Lane switching, jumping, sliding, collision, collecting, scoring	Achieved
Endless environment without growing memory use	Achieved through pooling and recycling
Progressive difficulty	Achieved through speed growth and pattern rules
Menu, pause, game-over and live score interface	Achieved
Audio and particle effects tied to gameplay	Achieved
Keyboard and touch support	Achieved

Chapter 15: Project Conclusion and Future Enhancements

15.1 Summary of Accomplishments

This project produced a complete 3D endless runner that demonstrates the main areas of game development: mechanics design, level generation, asset creation, scripting, user interface, audio and visual effects, testing and optimisation. The game needs no external assets, runs from a single script and can be extended with custom models, which makes it both a finished game and a reusable framework.

15.2 Implemented Mechanics

The implemented mechanics are three-lane movement, jumping with custom gravity, sliding, obstacle collision rules, coin collection, Magnet and Shield power-ups, enemy coyotes, speed-based difficulty, scoring with a saved best score, and pause, restart and mute controls.

15.3 Technical Challenges

- Designing obstacle patterns that are always survivable but still challenging.
- Keeping memory use stable in an endless world, which led to the pooling and recycling design.
- Making the controls feel responsive for both keyboard and touch input.
- Generating sound and models from code in a way that still looks and sounds good.
- Supporting replacement of generated models without breaking the game.

15.4 Limitations

- The game has one environment and one character.
- Collision is lane-based and does not model real physical contact.
- The visual style uses simple primitive shapes rather than detailed models.
- Scores are stored only on the local device.
- There is no tutorial level or in-game settings menu.

15.5 Learning Outcomes

The project gave practical experience in Unity scene construction, C# scripting, game state management, object pooling, procedural content generation, input handling for several devices, interface design, audio synthesis, particle systems, debugging and performance optimisation. It also showed how design choices, such as fair obstacle patterns and gradual difficulty, affect how a game feels to play.

15.6 Future Enhancements

Enhancement	Description
Multiplayer	Compare scores or race against other players, locally or online

Enhancement	Description
Advanced AI	Coyotes that predict the player's lane and adapt to their habits
More levels and themes	Additional environments such as snow, forest and city, which share the same engine
Mobile deployment	Build for Android and iOS with tuned touch controls and performance settings
Online leaderboards	Global scores and daily challenges
Character animation	Imported rigged characters with Animator Controllers
Shop and upgrades	Spend coins on characters, boards and power-up duration
AR and VR	A first-person VR runner or an AR mode that places the trail on a real surface

15.7 Conclusion

DesertRunner meets its goal of a complete, playable and extensible 3D game built with Unity and C#. It shows that a polished experience can come from simple, well-tuned mechanics and careful engineering, and it gives a strong base for the enhancements listed above.

References

1. Unity Technologies, Unity Manual and Scripting API, docs.unity3d.com.
2. Unity Technologies, Universal Render Pipeline documentation.
3. Schell, J., The Art of Game Design: A Book of Lenses.
4. Nystrom, R., Game Programming Patterns (Object Pool and State patterns).
5. Public information on Temple Run, Subway Surfers and related endless-runner games.